SIMATS ENGINEERING

CO1 - ASSESSMENT TOOL 3

Name of the Student	**U. Somesh Kumar**
Reg. No	**192425019**
GitHub Link	**https://github.com/someshkumar12345/Deep-Learning-lab-Experiments**

EXPERIMENTS

COURSE NAME: Deep Learning COURSE CODE: MLA0403 FACULTY : Dr. Arul Raj A.M

COURSE OUTCOME COVERED

CO 1: Learning Algorithms, Maximum Likelihood Estimation (MLE), Building Machine Learning Algorithms, Neural Networks, Artificial Neurons, Perceptron, Multilayer Perceptron (MLP), Forward Propagation, Backpropagation Algorithm, Gradient Descent, Stochastic Gradient Descent (SGD), Mini Batch Gradient Descent, Curse of Dimensionality. (BTL-4)

CO1 Weightage: 15%

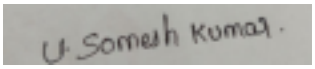
Assessment Tool Weightage: 35%

Duration: 30 minutes

Marks: 50

Code of Conduct:

I U. Somesh Kumar (192425019) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.



U. Somesh Kumar Signature of the student: Name of the student:

QUESTIONS:10 Total Marks:50 CO1- AT3 Experiments

Questions

1. Learning Algorithms

Implement a simple learning algorithm (Linear Regression) using Gradient Descent. Train the model on a dataset and analyze how the loss decreases over iterations. Plot the learning curve.

Answer :

Aim

To implement Linear Regression using Gradient Descent, train it on a dataset, and analyze the decrease in loss over iterations.

Algorithm

1. Import required libraries.
2. Generate/load dataset.
3. Initialize weight and bias.
4. Calculate predictions.
5. Compute Mean Squared Error (MSE).
6. Update parameters using Gradient Descent.
7. Repeat for specified iterations.
8. Plot the learning curve.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,2,3,4,5],dtype=float)
y = np.array([2,4,6,8,10],dtype=float)

w=0
b=0
lr=0.01
epochs=1000
losses=[]

n=len(x)

for i in range(epochs):
    y_pred=w*x+b

    dw=(-2/n)*np.sum(x*(y-y_pred))
    db=(-2/n)*np.sum(y-y_pred)

    w=w-lr*dw
    b=b-lr*db

    loss=np.mean((y-y_pred)**2)
    losses.append(loss)

plt.plot(losses)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Learning Curve")
plt.show()

print("Weight:",w)
print("Bias:",b)
```

Input:

Sample dataset (e.g., X = [1,2,3,4,5], Y = [2,4,6,8,10]), learning rate, epochs

Output

- **Final Weight**
- **Final Bias**
- **Learning Curve**

2. Maximum Likelihood Estimation (MLE)

Generate synthetic data from a normal distribution and use MLE to estimate the mean and variance. Compare estimated values with actual parameters.

Answer :

Aim

To implement Linear Regression using Gradient Descent, train it on a dataset, and analyze the decrease in loss over iterations.

Algorithm

1. Import required libraries.
2. Generate/load dataset.
3. Initialize weight and bias.
4. Calculate predictions.
5. Compute Mean Squared Error (MSE).
6. Update parameters using Gradient Descent.
7. Repeat for specified iterations.
8. Plot the learning curve.

Python Code

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1,2,3,4,5],dtype=float)
y = np.array([2,4,6,8,10],dtype=float)

w=0
b=0
lr=0.01
epochs=1000
losses=[]

n=len(x)

for i in range(epochs):
    y_pred=w*x+b

    dw=(-2/n)*np.sum(x*(y-y_pred))
    db=(-2/n)*np.sum(y-y_pred)

    w=w-lr*dw
    b=b-lr*db

    loss=np.mean((y-y_pred)**2)
    losses.append(loss)

plt.plot(losses)
plt.xlabel("Iterations")
plt.ylabel("Loss")
plt.title("Learning Curve")
plt.show()

print("Weight:",w)
print("Bias:",b)
```

Input:

Synthetic data generated from a normal distribution (Mean = 5, Variance = 4, Sample size = 1000)

Output

- Final Weight
- Final Bias
- Learning Curve

3. Building Machine Learning Algorithms

Build a complete machine learning model (data preprocessing, training, testing) using any classification dataset and evaluate performance using accuracy and confusion

matrix. **Answer : Aim**

To build a complete classification model and evaluate its performance.

Algorithm

1. Load dataset.
2. Split dataset.
3. Train classifier.
4. Predict output.
5. Calculate accuracy.
6. Display confusion matrix.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metricsimport accuracy_score,confusion_matrix

data=load_iris()

X_train,X_test,y_train,y_test=train_test_split(
data.data,data.target,test_size=0.2,random_state=42)

model=DecisionTreeClassifier()

model.fit(X_train,y_train)

pred=model.predict(X_test)

print("Accuracy:",accuracy_score(y_test,pred))
print("Confusion Matrix")
print(confusion_matrix(y_test,pred))
```

Input:

Iris dataset (or any classification dataset)

Output

Accuracy and Confusion Matrix.

4. Neural Networks

Q4. Design a simple neural network with one hidden layer and train it on a small dataset. Analyze how the network learns non-linear patterns.

Answer :

Aim

To design and train a simple neural network with one hidden layer.

Algorithm

1. Load dataset.
2. Split dataset.
3. Create neural network.
4. Train model.
5. Predict.
6. Evaluate accuracy.

Python Code

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier

iris=load_iris()
```

```
X_train,X_test,y_train,y_test=train_test_split(
iris.data,iris.target,test_size=0.2,random_state=1)

model=MLPClassifier(hidden_layer_sizes=(10,),max_iter=1000)

model.fit(X_train,y_train)

print("Accuracy:",model.score(X_test,y_test))
```

Input:
Iris dataset (training and testing data)
Output:
Model Accuracy

5. Artificial Neurons

Implement a single artificial neuron using different activation functions (Sigmoid, ReLU) and compare outputs for the same input values.

Answer :

Aim

To implement an artificial neuron using Sigmoid and ReLU activation functions. **Algorithm**

1. Define input values.
2. Calculate weighted sum.
3. Apply Sigmoid.
4. Apply ReLU.
5. Compare outputs.

Python Code

```
import numpy as np

x=np.array([1,2,3])

w=np.array([0.5,0.2,0.8])

z=np.dot(x,w)

sigmoid=1/(1+np.exp(-z))

relu=max(0,z)

print("Weighted Sum:",z)
print("Sigmoid:",sigmoid)
print("ReLU:",relu)
```

Input:
Input values (e.g., [1,2,3]) and weights (e.g., [0.5,0.2,0.8])
Output
Weighted Sum, Sigmoid Output and ReLU Output.

6. Perceptron and Multilayer Perceptron (MLP)

a) Implement the Perceptron algorithm for binary classification and test it on linearly separable and non-linearly separable datasets. Analyze the results.

Answer :

. Aim

To implement the Perceptron algorithm for binary classification.

Algorithm

1. Load dataset.
2. Train Perceptron.
3. Predict.
4. Evaluate accuracy.

Python Code

```
from sklearn.linear_model import Perceptron
from sklearn.datasetsimport make_classification
from sklearn.model_selection import train_test_split

X,y=make_classification(n_samples=100,n_features=2,
n_redundant=0,n_clusters_per_class=1)

X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2)

model=Perceptron()

model.fit(X_train,y_train)

print("Accuracy:",model.score(X_test,y_test))
```

Input:

Binary classification dataset (generated using make_classification)

Output

Classification Accuracy

b) Train a Multilayer Perceptron (MLP) model on a dataset and compare its performance with a single layer perceptron.

Answer :

Aim

To compare MLP with a single-layer perceptron.

Algorithm

1. Load dataset.
2. Train Perceptron.
3. Train MLP.
4. Compare accuracy.

Python Code

```
from sklearn.neural_network import MLPClassifier
from sklearn.linear_model import Perceptron
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

iris=load_iris()

X_train,X_test,y_train,y_test=train_test_split(
iris.data,iris.target,test_size=0.2)

p=Perceptron()
p.fit(X_train,y_train)
mlp=MLPClassifier(hidden_layer_sizes=(20,),max_iter=1000)
mlp.fit(X_train,y_train)

print("Perceptron:",p.score(X_test,y_test))
```

```
print("MLP:",mlp.score(X_test,y_test))
```

Input:
Iris Dataset
Output:
Comparison of Accuracies.

7. Forward Propagation and Backpropagation Algorithm

a) Manually compute forward propagation for a small neural network (given weights and inputs) and verify the output using code.

Answer :

Aim

To perform forward propagation manually and verify using code.

Algorithm

1. Define input.
2. Multiply by weights.
3. Add bias.
4. Apply activation.

Python Code

```
import numpy as np

x=np.array([1,2])

w=np.array([0.5,0.3])

b=0.1

z=np.dot(x,w)+b

output=1/(1+np.exp(-z))

print("Output:",output)
```

Input:
Inputs = [1,2], Weights = [0.5,0.3], Bias = 0.1
Output
Network Output.

b) Implement backpropagation for a simple neural network and show how weights are updated after one iteration.

Answer :

Aim

To implement one iteration of backpropagation.

Algorithm

1. Compute prediction.
2. Compute error.
3. Calculate gradient.
4. Update weights.

Python Code

```
w=2
x=4
target=10
lr=0.01
```

```
prediction=w*x

error=target-prediction

gradient=-2*x*error

w_new=w-lr*gradient

print("Updated Weight:",w_new)
```

Input:
Weight = 2, Input = 4, Target = 10, Learning rate = 0.01
Output:
Updated Weight.

8. Gradient Descent and Stochastic Gradient Descent (SGD)

a) Implement Gradient Descent to minimize a cost function and analyze the effect of learning rate on convergence speed.

Answer :

Aim

To minimize a cost function using Gradient Descent.

Algorithm

1. Initialize parameter.
2. Compute gradient.
3. Update parameter.
4. Repeat.

Python Code

```
x=10
lr=0.1

for i in range(20):
    grad=2*x
    x=x-lr*grad

print("Minimum Value:",x)
```

Input:
Initial parameter (x = 10), Learning rate = 0.1
Output:
Minimum parameter value after convergence.

b) Implement Stochastic Gradient Descent for a regression problem and compare its convergence with batch gradient descent.

Answer :

Aim

To implement SGD for regression and compare convergence.

Algorithm

1. Load dataset.
2. Train SGD Regressor.
3. Predict.
4. Evaluate.

Python Code

```
from sklearn.linear_model import SGDRegressor
```

```
from sklearn.datasets import make_regression

X,y=make_regression(n_samples=100,n_features=1)

model=SGDRegressor(max_iter=1000)

model.fit(X,y)

print("Coefficient:",model.coef_)
```

Input:

Regression dataset (make_regression)

Output

Regression Coefficient.

9. Mini-Batch Gradient Descent

Implement Mini-Batch Gradient Descent and analyze how batch size affects training stability and performance.

Answer :

Aim

To implement Mini-Batch Gradient Descent and study batch size effects.

Algorithm

1. Divide dataset into batches.
2. Train each batch.
3. Update weights.
4. Repeat until convergence.

Python Code

```
import numpy as np

data=np.arange(100)

batch_size=20

for i in range(0,len(data),batch_size):
   batch=data[i:i+batch_size]
   print(batch)
```

Input:

Dataset of 100 values, Batch size = 20

Output:

Mini-batches of the dataset.

10. Curse of Dimensionality

Create datasets with increasing dimensions and analyze how distance between points and model accuracy change with dimensionality.

Answer :

Aim

To analyze how increasing dimensions affect distances and model accuracy.

Algorithm

1. Generate datasets with different dimensions.

2. Compute pairwise distances.
3. Train classifier.
4. Compare accuracy.

Python Code:

```
from sklearn.datasets import make_classification
from sklearn.neighborsimport KNeighborsClassifier
from sklearn.model_selection import train_test_split

for dim in [2,5,10,20]:

    X,y=make_classification(
        n_samples=500,
        n_features=dim,
        n_informative=2,
        n_redundant=0,
        random_state=42
    )

    X_train,X_test,y_train,y_test=train_test_split(
        X,y,test_size=0.2)

    model=KNeighborsClassifier()

    model.fit(X_train,y_train)

    print("Dimension:",dim,
          "Accuracy:",model.score(X_test,y_test))
```

Input:

Datasets with dimensions 2, 5, 10, and 20

Output:

Accuracy for different dimensions, demonstrating the effect of the curse of dimensionality.